

Slot 4 · Sweep the asymmetry

File: `loss_sweep.py` · **Done when:** `uv run pytest tests/test_slot04.py` shows 3 passed · **Time:** 15 min

Today's idea

When the ETA is wrong, which mistake is worse?

- **Promise too short** → the food is late, the customer is angry, maybe a refund.
- **Promise too long** → the courier waits around, the customer may not order at all.

The **loss function** decides which mistake the model tries harder to avoid. Choosing it is a **business decision**, even though it looks like one number in the code.

Step 1 · Write the two counters

Open `loss_sweep.py`. Write two small functions. `pred` and `actual` are arrays of minutes.

function	meaning	returns
<code>promises_broken(pred, actual)</code>	share of orders where the promise was shorter than reality	a number from 0 to 1
<code>courier_idle_minutes(pred, actual)</code>	total minutes of waiting because the promise was too long	minutes (0 or more)

Check with a small example on paper: `pred = [30, 40, 50]`, `actual = [35, 35, 35]`.

- promises broken = 1 of 3 = **0.333**
- idle minutes = 0 + 5 + 15 = **20** (the short promise adds 0, not -5)

```
uv run pytest tests/test_slot04.py
```

Tip `np.clip(x, 0, None)` turns every negative number into 0.

Step 2 · Run the sweep

The training code is already written. It trains the same model 5 times, each time with a different **alpha** (how much the model fears being too short), and plots both counters.

```
uv run python loss_sweep.py
open logs/loss_sweep.png
```

It takes about 10–15 seconds. You should see:

alpha	promises_broken_pct	idle_min_per_1k	mae
0.5	76.5	725	6.18
0.6	67.3	1140	5.65
0.7	56.6	1762	5.35
0.8	43.3	2760	5.43
0.9	25.9	4850	6.52

Step 3 · Pick an alpha and defend it

In your team, choose one alpha. Be ready to explain **why** in one sentence.

- "0.8 — a broken promise costs us a refund and a bad rating."
- "0.6 — couriers are our scarcest resource; idle time means fewer deliveries."

There is no correct answer. That is the point.

Think At $\alpha = 0.5$ the model should break about **half** its promises. It breaks **76.5%**. Why? (Hint: the test data is from the *future*. Did something change in the world? We come back to this in Slot 12.)

If something goes wrong

what you see	what to do
both counters go up together	<code>courier_idle_minutes</code> is counting negative waiting. Clip at 0.
<code>promises_broken</code> is bigger than 1	You returned a count. Use <code>.mean()</code> , not <code>.sum()</code> .
the plot does not open	The file is at <code>logs/loss_sweep.png</code> . Open it from Finder.
it is very slow	Close other heavy apps. It should take under 20 seconds.